

运行时行为差异差分测试基础设施

目标

该基础设施主要是为了自动化地、可靠地比较编译器抽象间的差异，包括但不限于 `solc` 和 `solang`。

目前使用的工作流大概是：

1. 给定一个 solidity 源程序，或基于表达式生成。
2. 使用 `solc` 和 `solang` 编译。
3. 如果编译均成功，运行二者产物，比较其运行时行为。

工具链

在当前的稳定版本的 `solang` 中，绝大部分 solidity 特性无法在 `solang compile -target evm` 中使用，因此可以认为 `solang` 的 `evm` 目标不可用。因此设计使用 `polkadot` 去执行相应生成的 WASM 二进制。

由于模拟整个在链行为较为困难，该设计选择采取一种基于计算的解决方案。在目前的实现下表现为构造一个 `uint256 → uint256` 的纯函数，然后在其中嵌入可求值的表达式。

IO 循环测试

给定如下 solidity 代码：

```
contract GeneratedIoOracle {
    function run(uint256 a) external pure returns (uint256) {
        return a + 7;
    }
}
```

我们可以使用

```
uv run solidity-diff-fuzz io-diff --expression 'a + 7' --input 35 --
solang-target polkadot
```

进行测试。

限制

目前没有设计相对完整的计算运转逻辑，理论上可以将绝大部分测试转换为对某一数值进行计算（例如哈希异或等），

从而将其集成到当前测试逻辑中。但是这种测试方法是否能够反向识别错误内容仍有待考量。

同时目前没有考虑合约间交互，包括但不限于事件、交易执行、账户等。